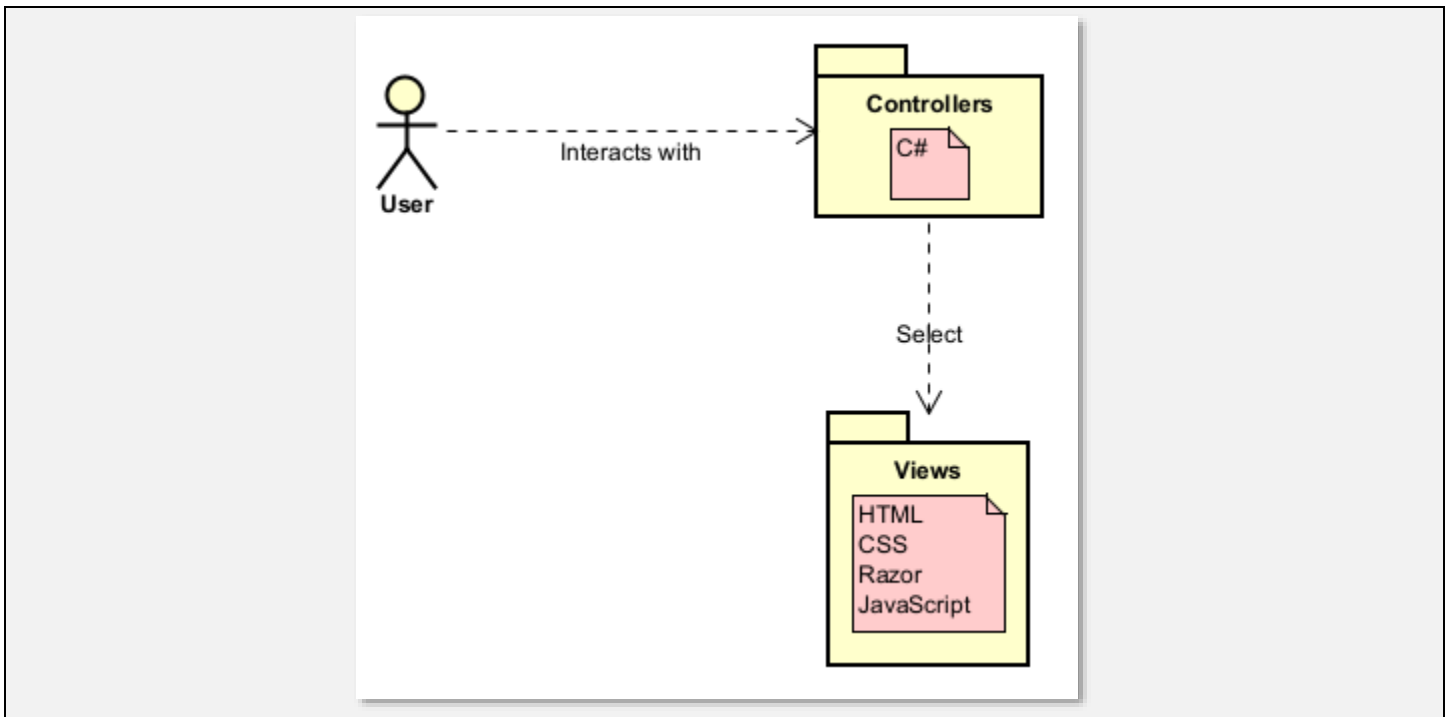


Architectural Overview



Controller Classes

- ❖ Are classes that typically inherit from the Controller class (Assembly: `Microsoft.AspNetCore.Mvc`)

```
public class HomeController : Controller
{
    // Fields, Properties, Action methods
}
```

- ❖ Each controller class must end with 'Controller.'
- E.g., `HomeController`
- ❖ URLs are mapped to controller classes.
 - E.g., the URL: `https://<host>/home/` maps to class `HomeController`
- ❖ The controller is responsible for processing the request.
 - They handle user input and interactions by calling the appropriate application logic.
 - They typically call a separate view component that generates the HTML markup.
- ❖ Public methods of controller classes are **action methods**.
 - Action methods contain logic for mapping a request to a business concern.
 - Business concerns should typically be represented as services that the controller accesses through dependency injection.

Routing

- ❖ Routing matches a request URL to an action on a controller.
- ❖ The routing middleware decides what to do with the incoming request.
 - Enables the use of URLs that do not map to a specific file.
 - Maps URL patterns to controllers
 - Utilizes a route map to know where to route requests.
- ❖ Types of routing
 - Conventional
 - Attribute (We'll cover this in the Web API lecture)

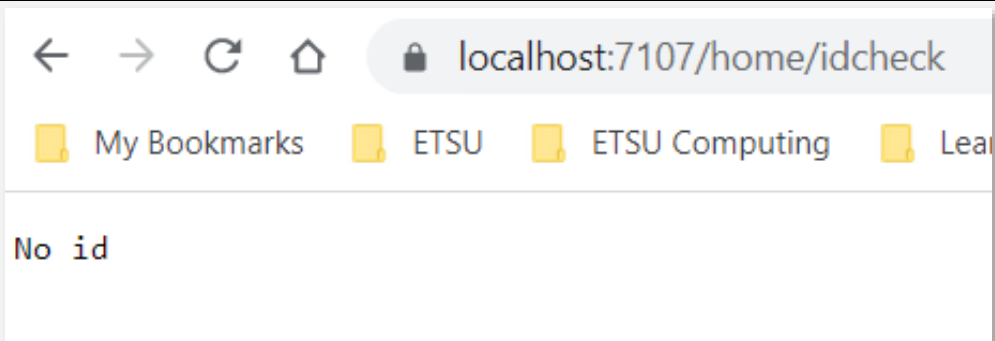
Endpoint Routing

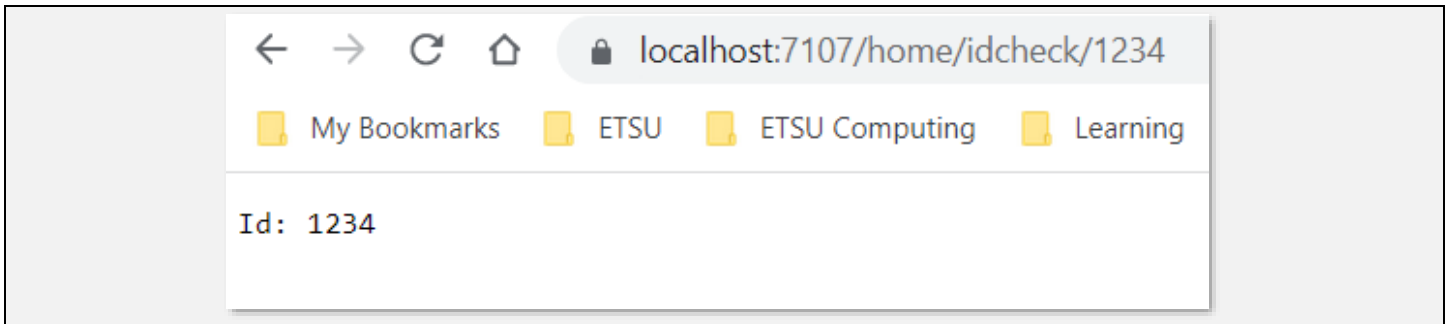
- ❖ An endpoint defines an operation that will handle and process incoming requests.
- ❖ Endpoint routing integrates with the middleware
 - **UseRouting** – adds route matching to the middleware pipeline. It must come before any route-aware middleware such as authorization
 - **MapControllerRoute** is used to create a single route.

```
app.UseRouting();  
...  
app.MapControllerRoute(...)
```

- ❖ Default routing allows us to have URLs in the form `/<controller>/<action>/<id>`.
E.g., `/home/index/1234`
- ❖ Here is a simple action method in the home controller.

```
public IActionResult IdCheck(string? id)  
{  
    var model = "No id";  
    if (id != null)  
    {  
        model = $"Id: {id}";  
    }  
    return Content(model);  
}
```





Conventional Routing

- ❖ Establishes a **convention** for URL paths
- ❖ Example of registering the 'default' route

```
app.MapControllerRoute(  
    name: "default",  
    pattern: "{controller=Home}/{action=Index}/{id?}")
```

- ❖ **{controller=Home}/{action=Index}/{id?}**
 - 3 segment URL e.g., /home/index/1234
 - {controller}: Matches any controller
 - {controller=Home}: Matches any controller but defaults to HomeController
 - {action}: Matches any action method
 - {action=Index}: Matches any action method but defaults to Index()
 - {id}: Matches to required parameter 'id' which is passed to the action method.
 - {id?}: Matches to an optional (the ?) parameter 'id' which is passed to the action method.
 - Any three segment URL (e.g., /products/details/5) will match this pattern
 - So, from the example:
 - {controller} = *products* = ProductController class
 - {action} = *details* = Details() method of the ProductController class
 - {id} = 5 = parameter passed to the Details() method

URL Patterns

- ❖ Define how URLs are parsed into segments and placeholders
- ❖ Route parameters are enclosed in braces: {controller}, {action}, {id}
- ❖ URL segments are separated by forward slashes (/)
- ❖ When a request is made, placeholders are replaced with actual values from the URL
- ❖ These values are then provided to the appropriate controller action
- ❖ **Example:** /Products/Details/5 matches pattern /{controller}/{action}/{id}
 - controller = "Products"
 - action = "Details"
 - id = "5"

Route Definition	Example of matching URL
<code>{controller}/{action}/{id}</code>	<code>/products/show/beverages</code> <code>/item/edit/1234</code>
<code>{controller}/{action}/{code}/{idnumber}</code>	<code>/thing/delete/E/4567</code>

❖ These routes are created with:

```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapControllerRoute(
    name: "fourSegments",
    pattern: "{controller}/{action}/{code}/{idnumber}");
```

Parameter Binding

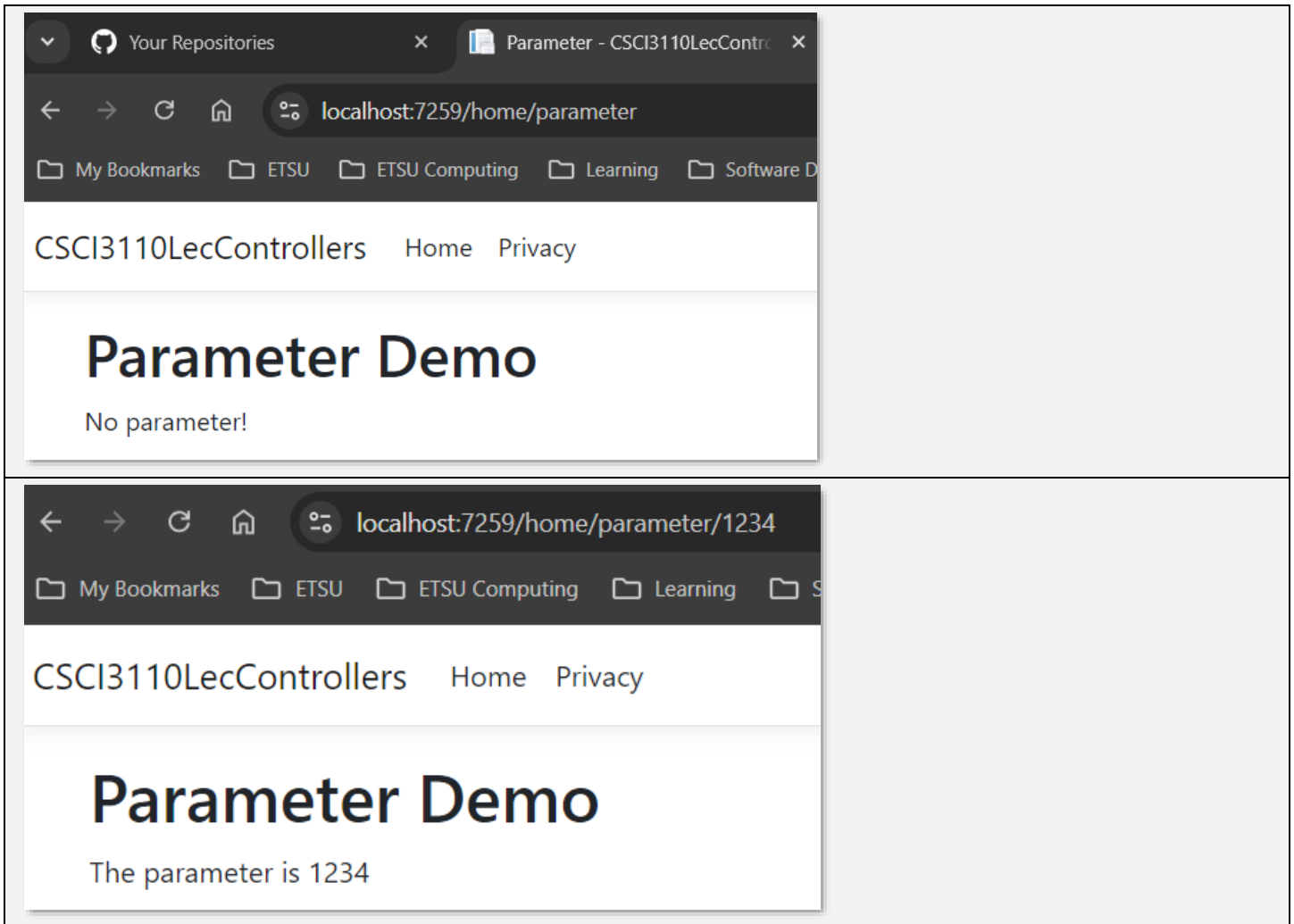
❖ The names used in the pattern should match the names used in the parameter list. Casing is ignored.

```
pattern: "{controller=Home}/{action=Index}/{id?}"
```

```
public IActionResult Parameter(string? id)
{
    ViewData["id"] = id;
    return View();
}
```

```
@{
    string? id = (string?)ViewData["id"];
    ViewData["Title"] = "Parameter";
}
```

```
<h1>Parameter Demo</h1>
@if(id != null)
{
    <p>The parameter is @id</p>
}
else
{
    <p>No parameter!</p>
}
```



❖ Same for four segments

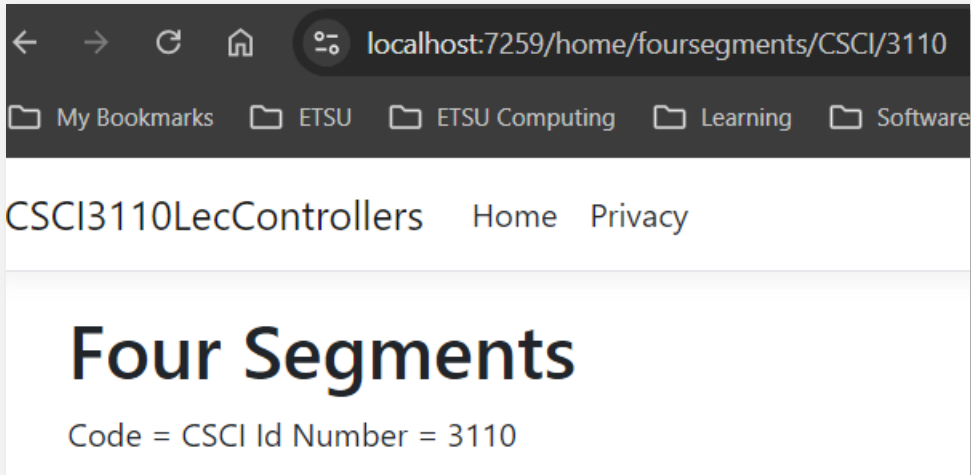
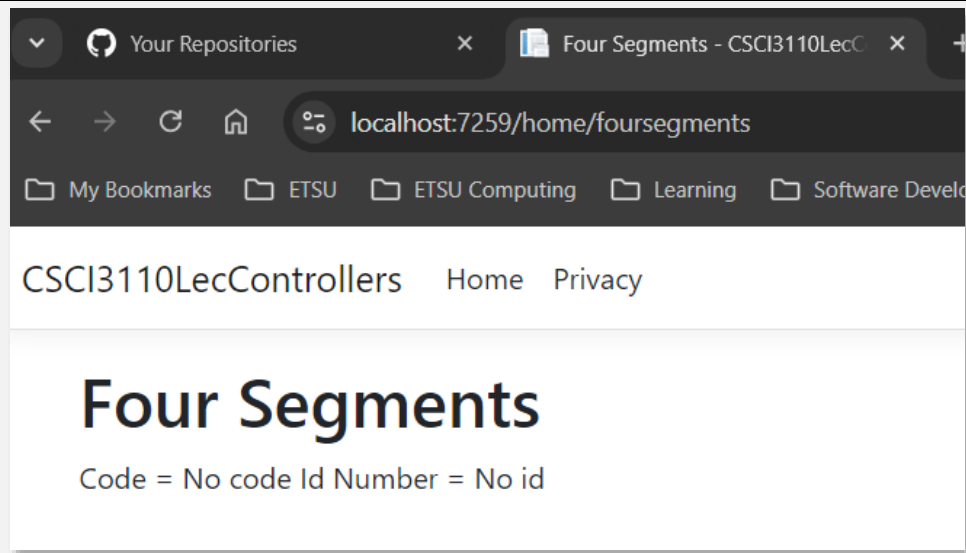
```
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapControllerRoute(
    name: "4segments",
    pattern: "{controller}/{action}/{code}/{idnumber}");

public IActionResult FourSegments(string code, string idNumber)
{
    ViewData["code"] = code;
    ViewData["idNumber"] = idNumber;
    return View();
}
```

```
@{
    string? code = (String?)ViewData["code"]??"No code";
    string? idNumber = (String?)ViewData["idNumber"]??"No id";
    ViewData["Title"] = "Four Segments";
}

<h1>@ViewData["Title"]</h1>

<p>Code = @code Id Number = @idNumber</p>
```



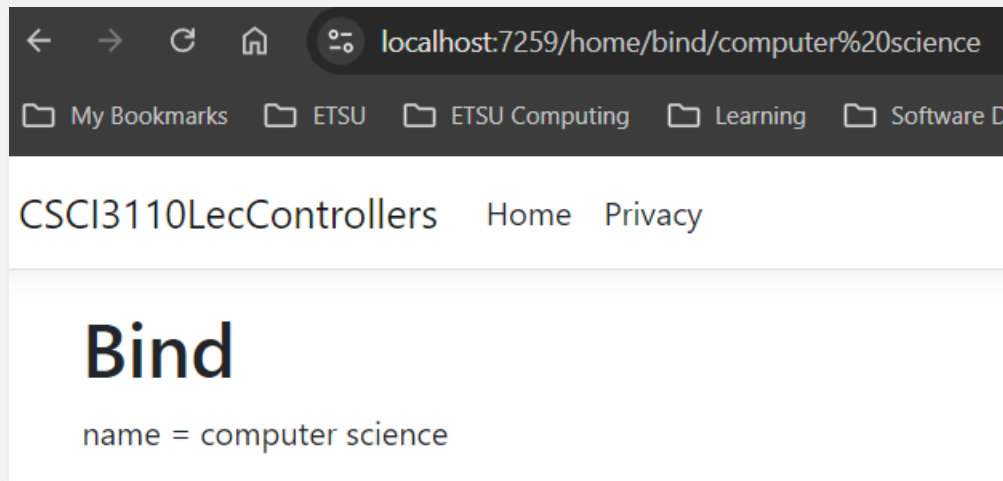
❖ You can use the Bind() attribute to rename parameters

```
pattern: "{controller=Home}/{action=Index}/{id?}"
public IActionResult Bind([Bind(Prefix = "id")] string? name)
{
    ViewData["name"] = name;
    return View();
}
```

```
@{
    string? name = (String?)ViewData["name"] ?? "No name";
    ViewData["Title"] = "Bind";
}

<h1>@ViewData["Title"]</h1>

<p>name = @name</p>
```



Custom Routes

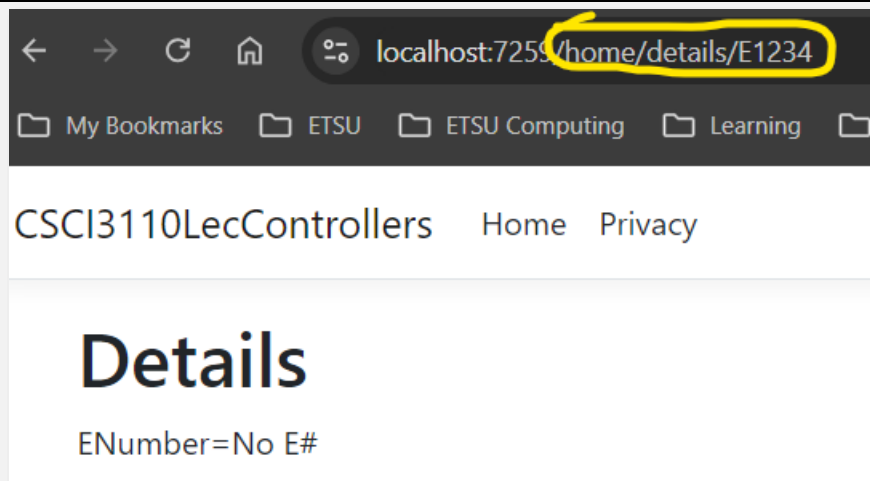
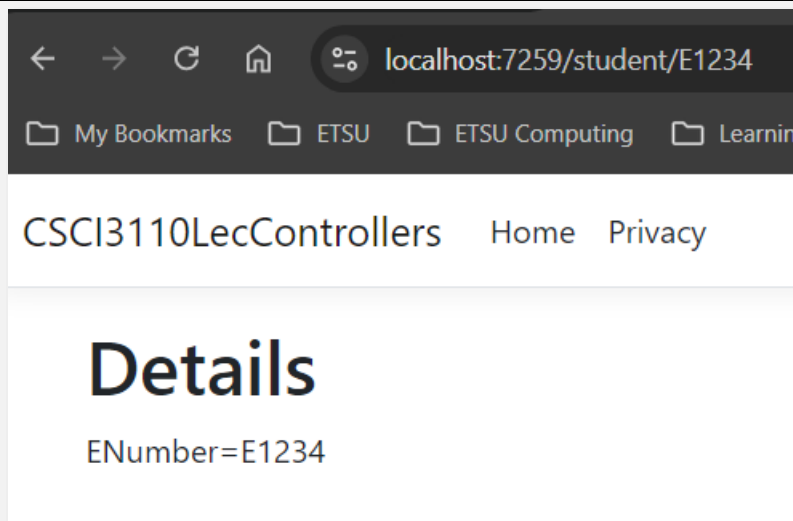
- ❖ Custom routes can contain literal values (e.g. student)
- ❖ You need to explicitly specify the placeholders
- ❖ E.g., Pattern: student/{enumber}
 - Matches 2 segment URL that starts with student
 - {enumber} matches a parameter
 - Need an anonymous object to specify the controller and the action method
 - new { controller = "Home", action = "Details" }
- ❖ **Tip: when arranging your routes put the more specific ones first**

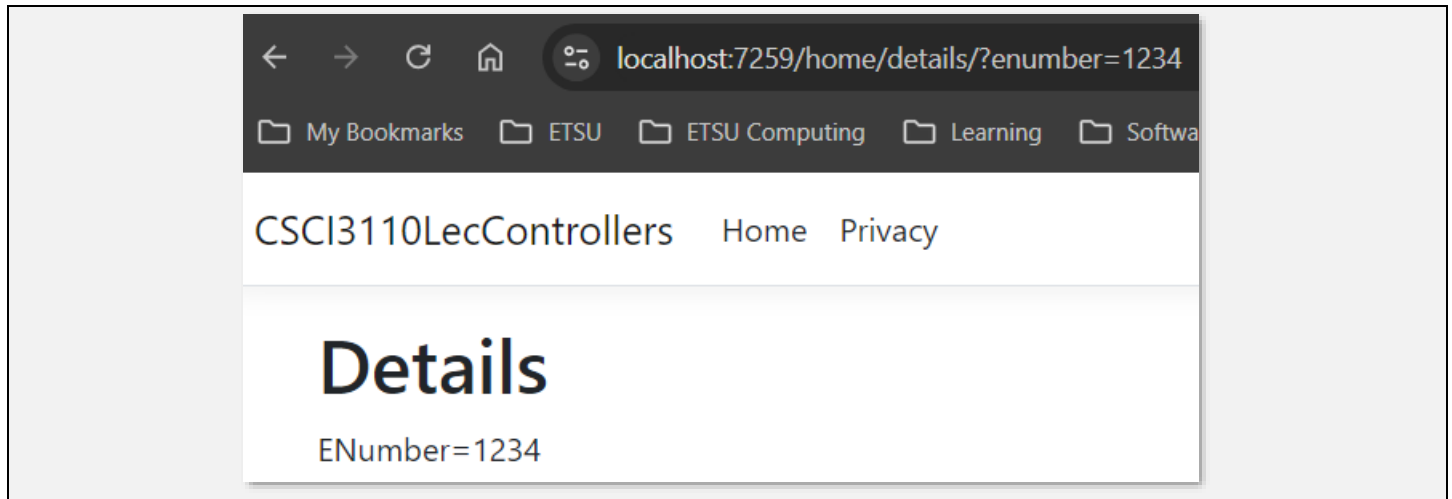
```
app.MapControllerRoute(
    name: "student",
    pattern: "student/{enumber}",
    defaults: new { controller = "Home", action = "Details" });
app.MapControllerRoute(
    name: "default",
    pattern: "{controller=Home}/{action=Index}/{id?}");
app.MapControllerRoute(
    name: "4segments",
    pattern: "{controller}/{action}/{code}/{idnumber}");
```

```
public IActionResult Details(string? enumber)
{
    ViewData["enumber"] = enumber;
    return View();
}

@{
    string? enumber = (String?)ViewData["enumber"] ?? "No E#";
    ViewData["Title"] = "Details";
}

<h1>@ViewData["Title"]</h1>
<p>ENumber=@enumber</p>
```





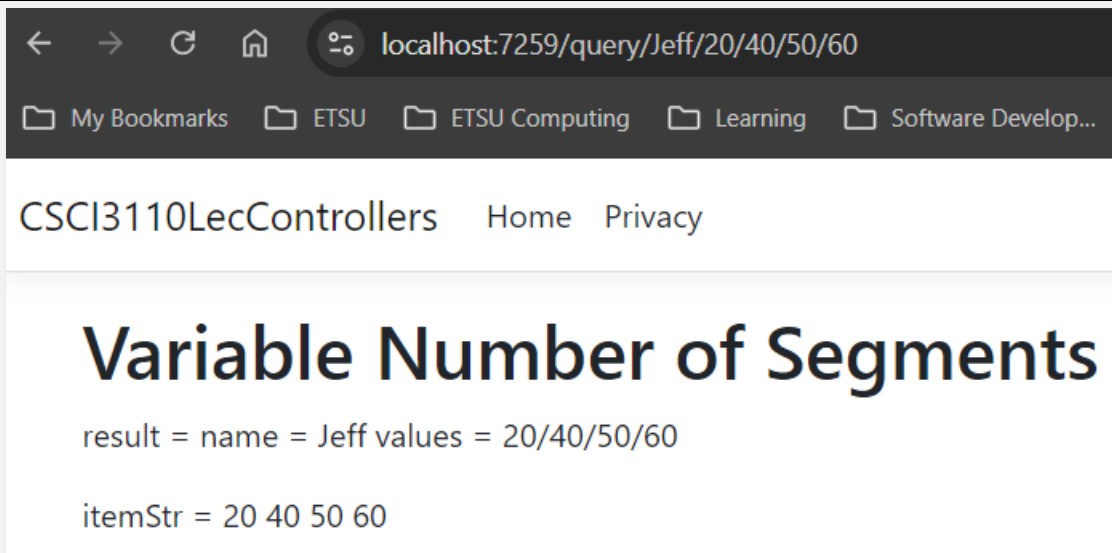
Variable Number of Segments in a URL Pattern

- ❖ Mark the last parameter with an asterisk (*)
 - E.g., `query/{name}/{*values}`

URL	Parameter Values
<code>/query/select/bikes/onsale</code>	name = "select" values = "bikes/onsale"
<code>/query/select/bikes</code>	name = "select" values = "bikes"
<code>/query/select</code>	name = "select" values = ""
<pre>app.MapControllerRoute(name: "variable", pattern: "query/{name}/{*values}", defaults: new { controller = "Home", action = "Variable" }); app.MapControllerRoute(name: "student", pattern: "student/{enumber}", defaults: new { controller = "Home", action = "Details" }); app.MapControllerRoute(name: "default", pattern: "{controller=Home}/{action=Index}/{id?}"); app.MapControllerRoute(name: "4segments", pattern: "{controller}/{action}/{code}/{idnumber}");</pre>	
<pre>public IActionResult Variable(string? name, string? values) { ViewData["result"] = \$"name = {name} values = {values}"; string[] itemArray = (values != null)?values.Split("/"):[]; ViewData["itemStr"] = String.Join(" ", itemArray); return View(); }</pre>	

```
@{
    var result = (String?)ViewData["result"] ?? "No result";
    var itemStr = (String?)ViewData["itemStr"] ?? "No item string";
    ViewData["Title"] = "Variable";
}

<h1>@ViewData["Title"] Number of Segments</h1>
<p>result = @result</p>
<p>itemStr = @itemStr</p>
```



Action Methods

- ❖ Handle requests from clients
- ❖ Handle **GET** (HttpGet) requests by default, to change the request type simply add the appropriate data annotation.

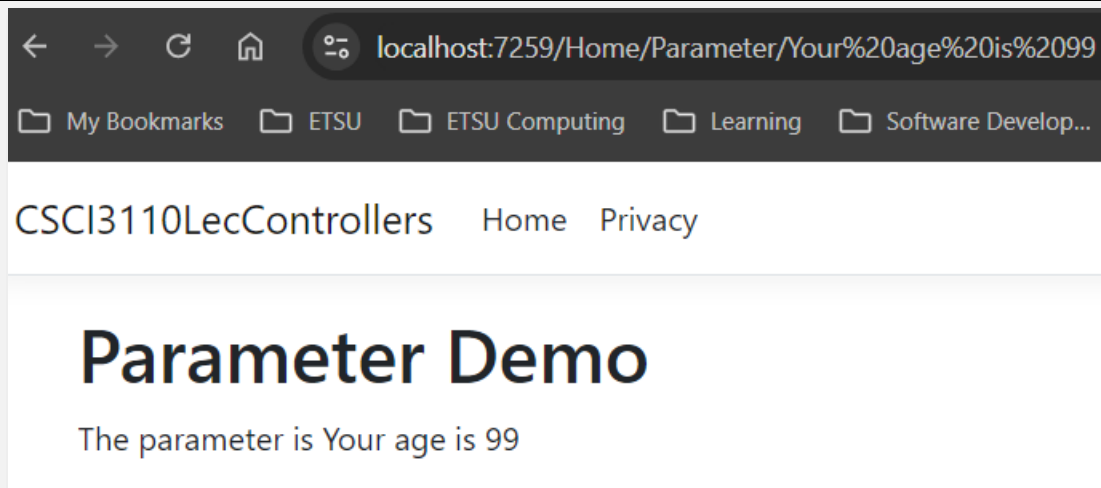
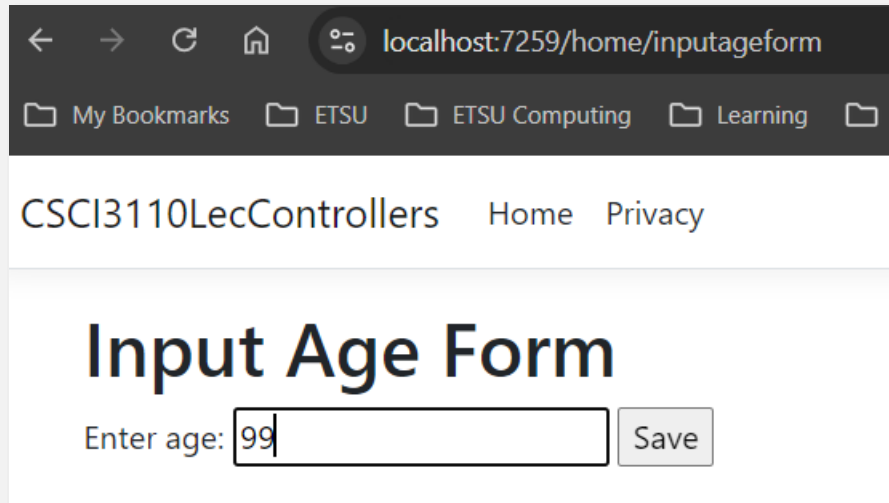
```
public IActionResult InputAgeForm()
{
    return View();
}

@{
    ViewData["Title"] = "Input Age Form";
}

<h1>@ViewData["Title"]</h1>

<form action="FormProcessor" method="post">
    <label>Enter age:</label>
    <input name="age" />
    <input type="submit" value="Save" />
</form>
```

```
[HttpPost]
public IActionResult FormProcessor(int age)
{
    return RedirectToAction("Parameter", new { id = $"Your age is {age}" });
}
```



Action Results

- ❖ Are returned from action methods.
- ❖ 'IActionResult' interface is the base interface for all action results.
- ❖ You decide which type of action result to return based on the task that the action method is performing.
- ❖ Form the response to a request.

```
public IActionResult Index()
{
    return View();
}
```

Action Result	Factory Method	Description
ViewResult	View()	Renders a view as a Web page.
PartialViewResult	PartialView()	Renders a section of a view that can be rendered inside another view.
RedirectResult	Redirect()	Redirects to another action method by using its URL.
RedirectToRouteResult	RedirectToAction() RedirectToRoute()	Redirects to another action method.
ContentResult	Content()	Returns a user-defined content type.
JsonResult	Json()	Returns a serialized JSON object.
FileResult	File()	Returns binary output to write to the response.

Using a few Factory Methods with Examples

```
return RedirectToAction("Index", "Home");
```

Redirect to the Index action method of the HomeController.

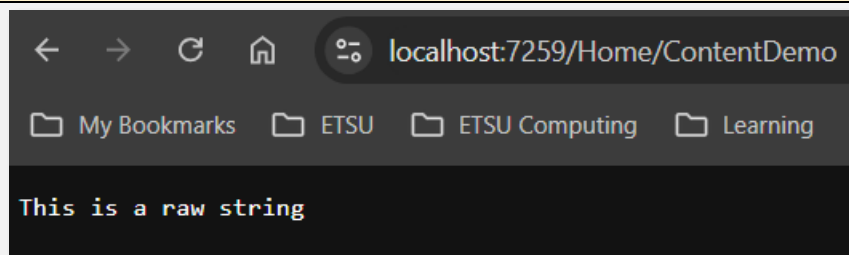
```
return RedirectToAction("Enroll", "Student", new {id = studentId});
```

Redirects to the Enroll action method of the StudentController with route parameter id.

```
return Content("This is a raw string");
```

Responds with a raw string

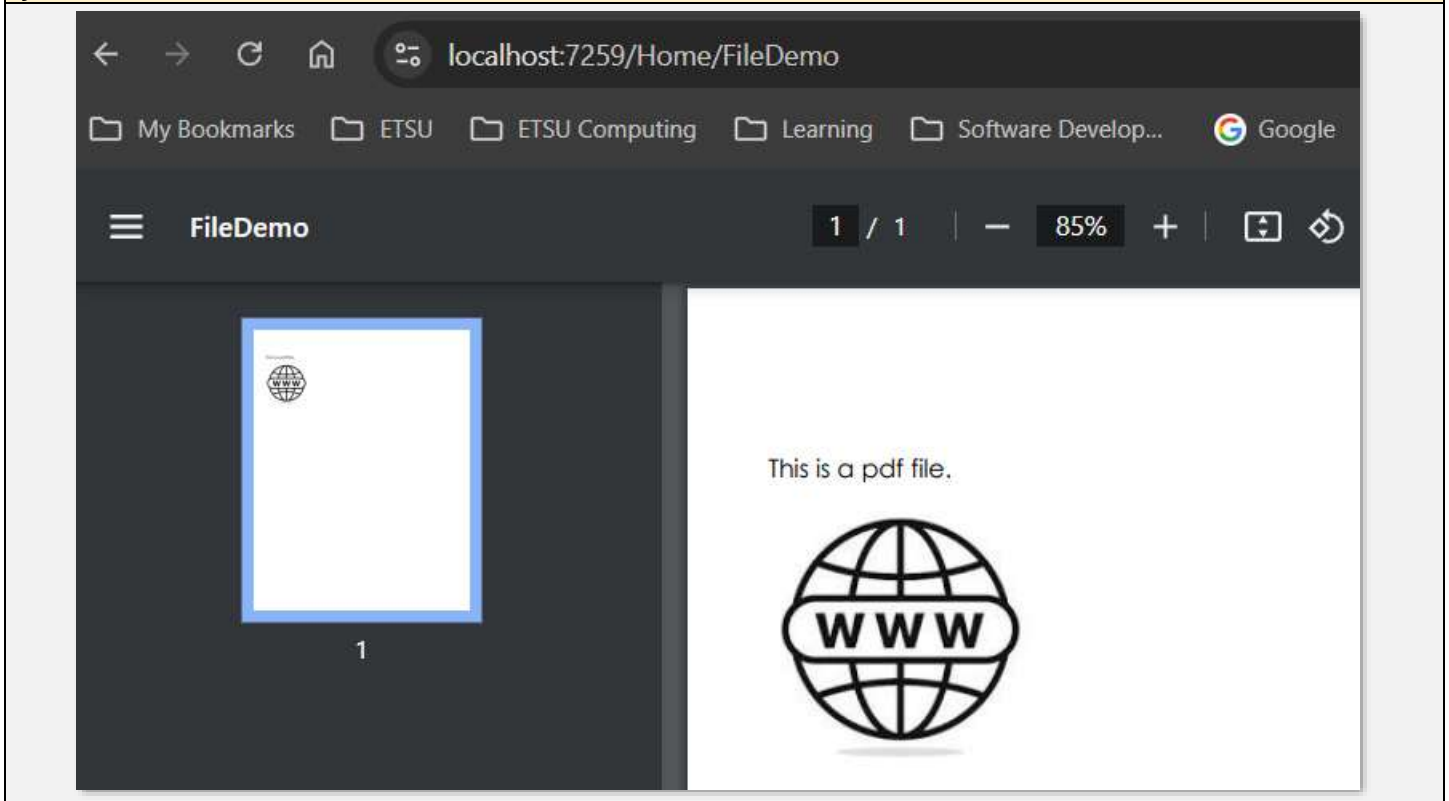
```
public IActionResult ContentDemo()
{
    return Content("This is a raw string");
}
```



```
return File("myfile.pdf", "application/pdf");
```

Returns binary representation of *myfile.pdf*. Notice the **application/pdf** MIME type. In this example, the file is in the *wwwroot* folder.

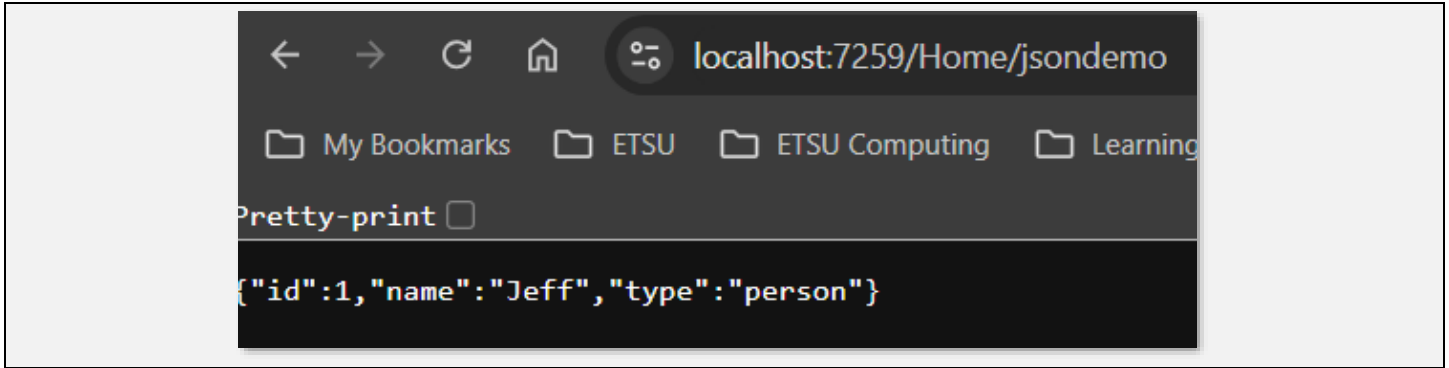
```
public IActionResult FileDemo()  
{  
    return File("myfile.pdf", "application/pdf");  
}
```



```
return Json(new { id = 1, name = "Jeff", type = "person" });
```

Returns the given objects as a JSON object.

```
public IActionResult JsonDemo()  
{  
    return Json(new { id = 1, name = "Jeff", type = "person" });  
}
```



Download to Client

1. The server generates the file content and sends it to the browser.
2. The browser triggers a download prompt.
3. The user chooses where to save the file on their local machine.
4. The file ends up stored client-side (on the user's computer).

```
public IActionResult ClientSideSave()
{
    return View();
}
```

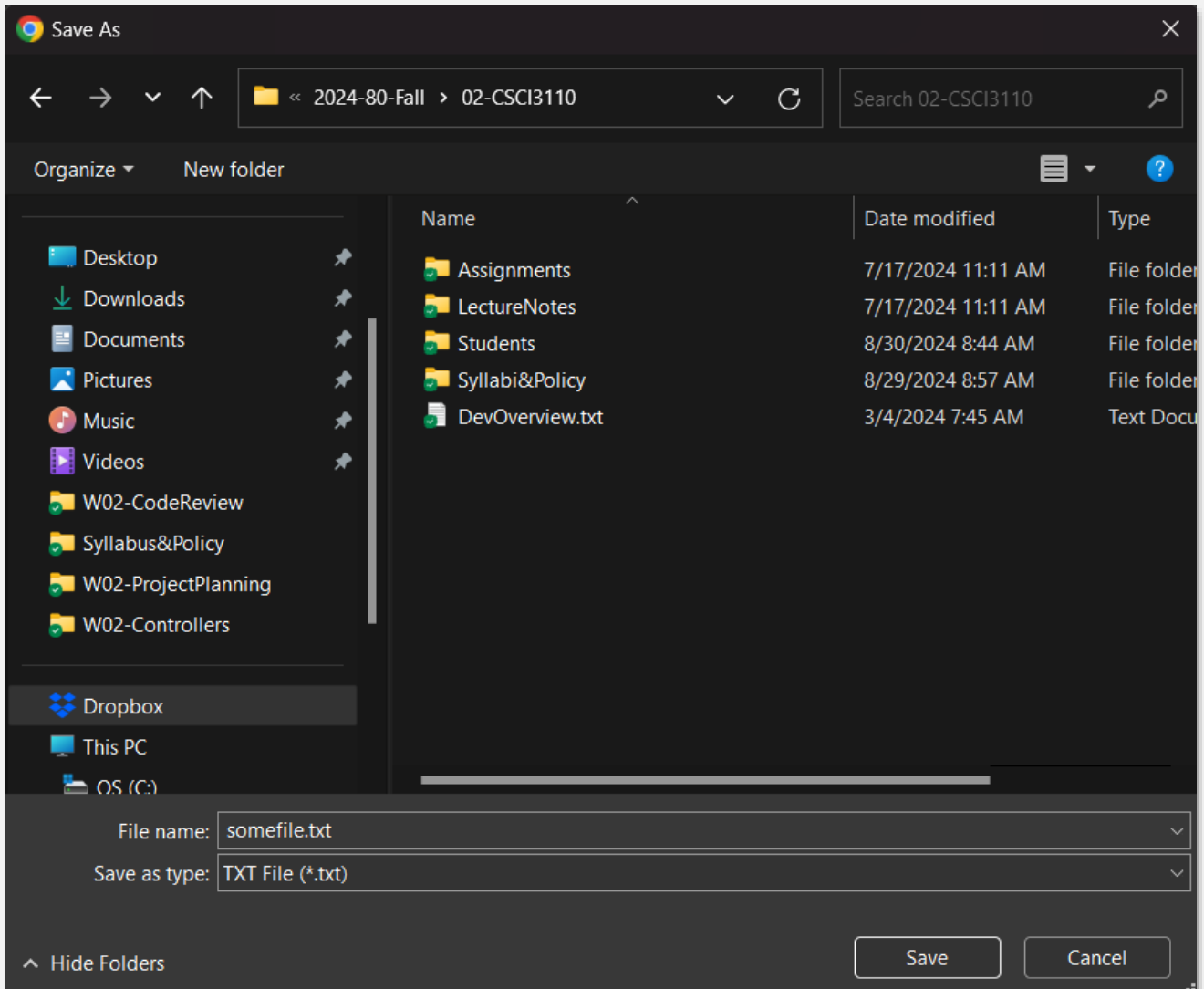
```
@{
    ViewData["Title"] = "Client-Side Save";
}

<div class="text-center">
    <h1 class="display-4">@ViewData["Title"]</h1>
    <a asp-action="FileSaveDemo">File Save Demo</a>
</div>
```

```
public IActionResult FileSaveDemo()
{
    // Create a memory stream to stream the data
    string stringToSave = "This is the data to save!\n";
    byte[] bytesToSave = Encoding.UTF8.GetBytes(stringToSave);
    MemoryStream ms = new();
    ms.Write(bytesToSave, 0, bytesToSave.Length);
    ms.Position = 0;
    // Download the memory stream
    string path = "somefile.txt";
    return File(ms, "application/octet-stream", Path.GetFileName(path));
}
```

Client-Side Save

[File Save Demo](#)



File Upload

```
public IActionResult FileUploadDemo()
{
    return View();
}
```

```
@{  
    ViewData["Title"] = "File Upload Demo";  
}  
  
<h1>@ViewData["Title"]</h1>  
  
<form asp-action="DoUpload" method="post" enctype="multipart/form-data">  
    <label for="files">Choose the file to upload</label>  
    <input id="files" name="files" type="file"/>  
    <input type="submit" value="Upload"/>  
</form>
```

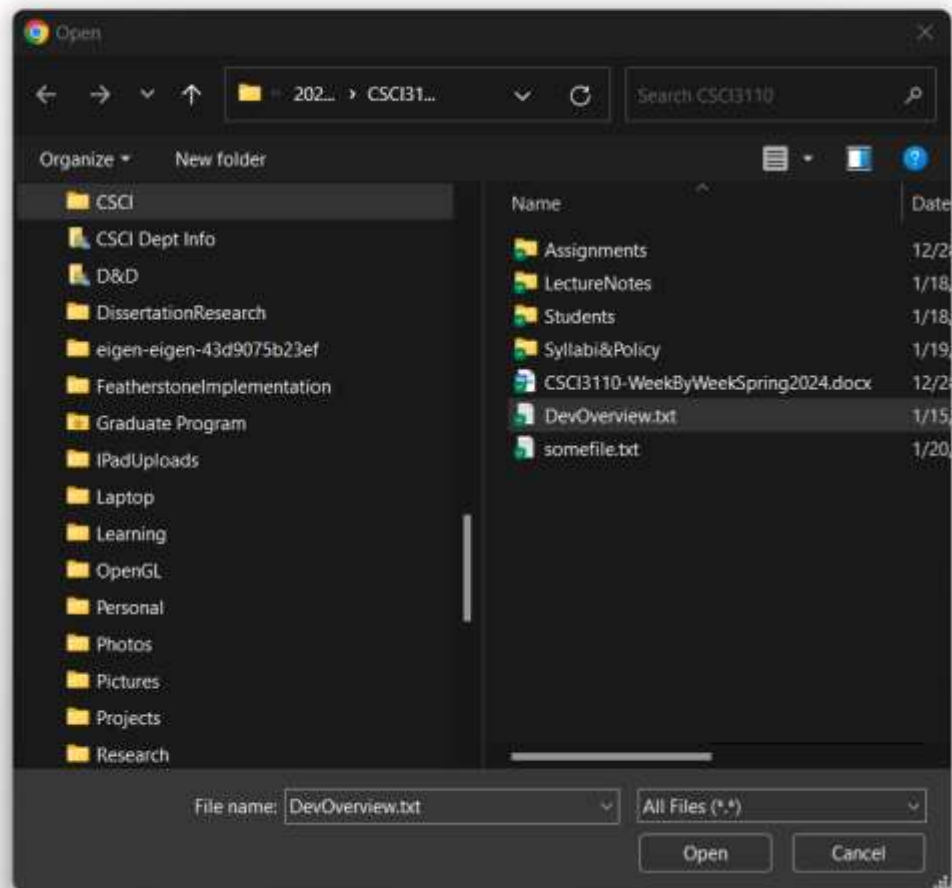
File Upload Demo

Choose the file to upload

Choose File

No file chosen

Upload



File Upload Demo

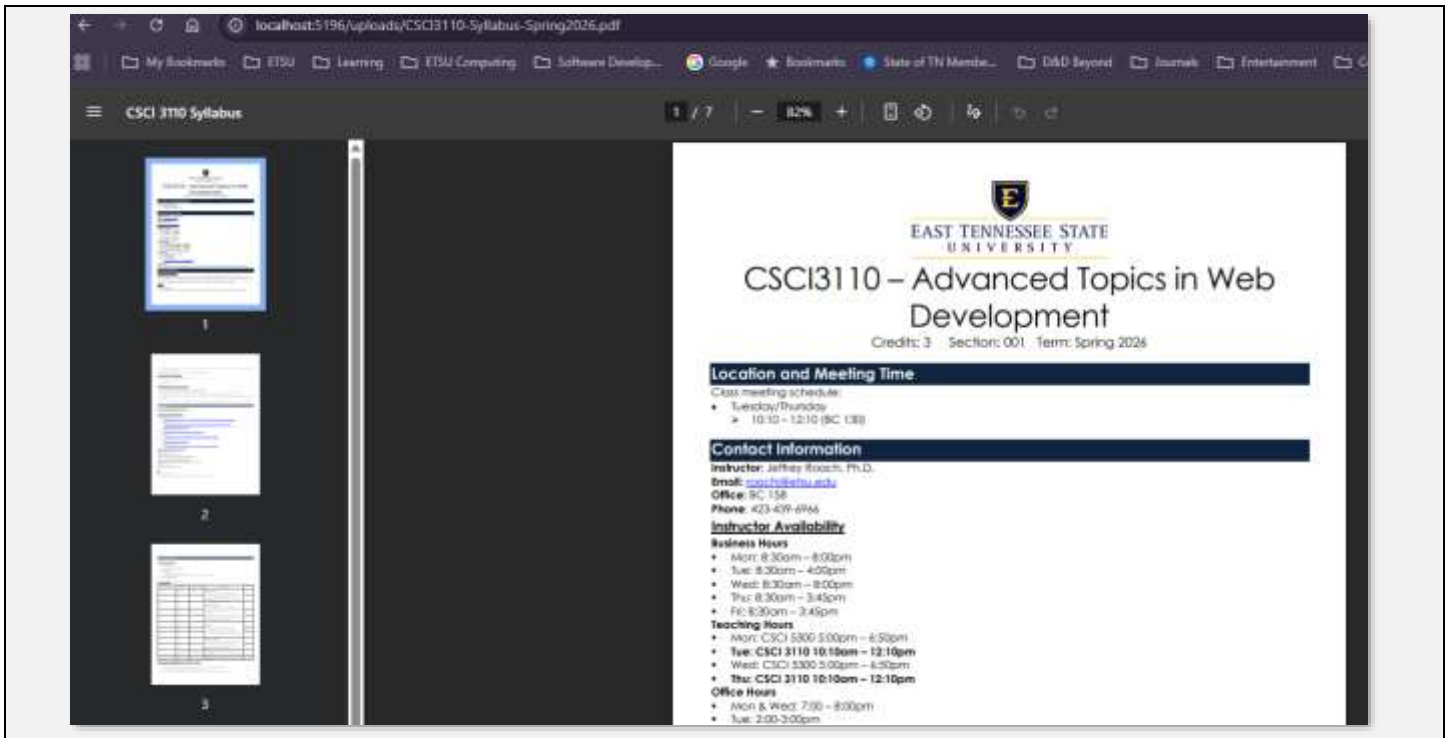
Choose the file to upload

Choose File

DevOverview.txt

Upload

```
[HttpPost]
public async Task<IActionResult> DoUpload(IList<IFormFile> files)
{
    // Get the first file in the list.
    IFormFile? file = files.FirstOrDefault();
    if (file == null || file.Length <= 0)
    {
        return BadRequest("No file selected");
    }
    // Define where to save
    string uploadsFolder = Path.Combine(Directory.GetCurrentDirectory(), "wwwroot",
"uploads");
    Directory.CreateDirectory(uploadsFolder); // Create folder if it doesn't exist
    string filePath = Path.Combine(uploadsFolder, file.FileName);
    _logger.Log(LogLevel.Information, filePath);
    // Save the file
    using (var stream = new FileStream(filePath, FileMode.Create))
    {
        await file.CopyToAsync(stream);
    }
    return Redirect($"/uploads/{file.FileName}");
}
```



Accessing the Application Settings

- ❖ The application settings is typically stored in an ASP.NET Configuration file with the name `"appsettings.json"`.
- ❖ Here is the initial generated start file:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*"
}
```

Let us add the application name to this file:

```
{
  "Logging": {
    "LogLevel": {
      "Default": "Information",
      "Microsoft": "Warning",
      "Microsoft.Hosting.Lifetime": "Information"
    }
  },
  "AllowedHosts": "*",
  "AppName": "My most excellent App!"
}
```

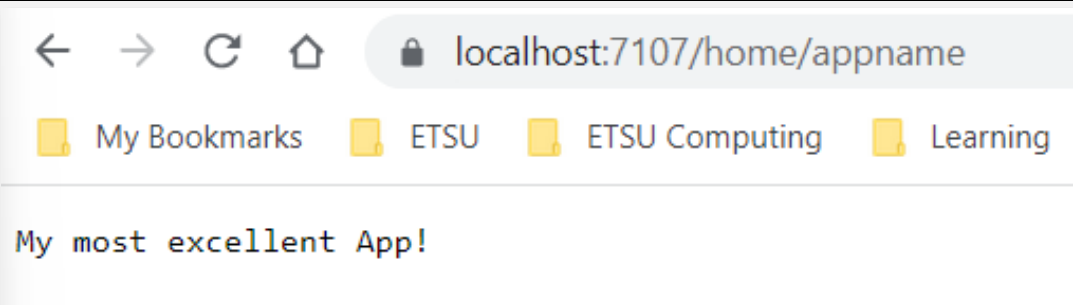
- ❖ The Startup class is already configured to access the configuration
- ❖ We can then **inject** the configuration into the home controller through the constructor:

```
public class HomeController : Controller
{
    private readonly IConfiguration _configuration;
    private readonly ILogger<HomeController> _logger;

    public HomeController(IConfiguration configuration, ILogger<HomeController>
logger)
    {
        _configuration = configuration;
        _logger = logger;
    }

    ...

    public IActionResult AppName()
    {
        string? appname = _configuration.GetValue<string>("AppName") ?? "No name";
        return Content(appname);
    }
}
```



Using the Logger

- ❖ Notice that a logger is injected into the HomeController

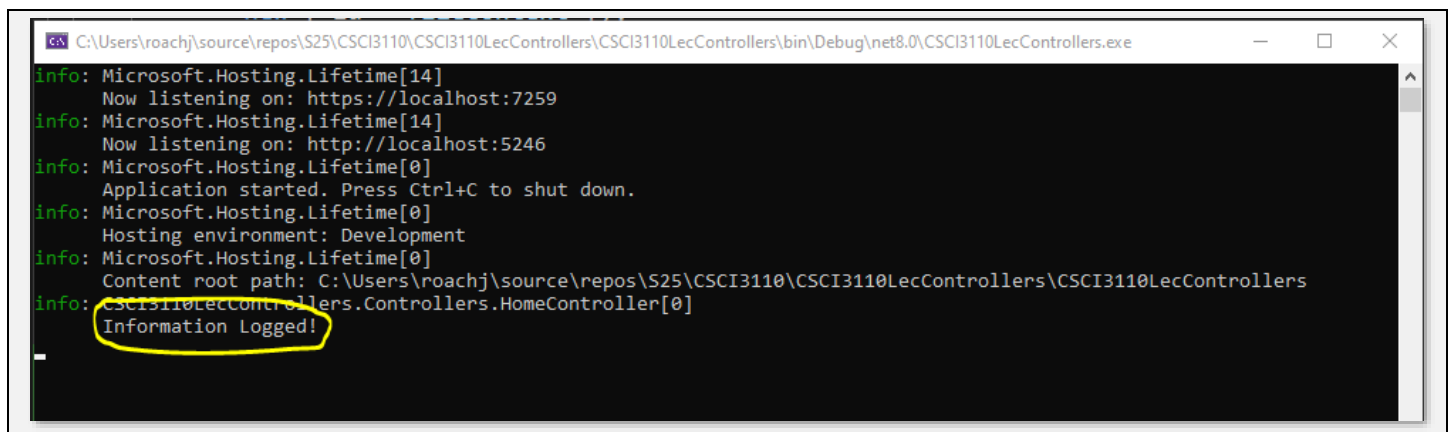
```
public class HomeController : Controller
{
    private readonly ILogger<HomeController> _logger;

    public HomeController(ILogger<HomeController> logger)
    {
        _logger = logger;
    }
}
```

❖ You can send information to the logger like so:

```
public IActionResult ShowALog()
{
    _logger.LogInformation("Information Logged!");
    return Content("Logger Shown");
}
```

❖ When the application is running, you can view the logged information in console.



```
C:\Users\roachj\source\repos\S25\CSCI3110\CSCI3110LecControllers\CSCI3110LecControllers\bin\Debug\net8.0\CSCI3110LecControllers.exe
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: https://localhost:7259
info: Microsoft.Hosting.Lifetime[14]
      Now listening on: http://localhost:5246
info: Microsoft.Hosting.Lifetime[0]
      Application started. Press Ctrl+C to shut down.
info: Microsoft.Hosting.Lifetime[0]
      Hosting environment: Development
info: Microsoft.Hosting.Lifetime[0]
      Content root path: C:\Users\roachj\source\repos\S25\CSCI3110\CSCI3110LecControllers\CSCI3110LecControllers
info: CSC3110LecControllers.Controllers.HomeController[0]
      Information Logged!
```

Bibliography

- Nowak, R., Larkin, K., & Anderson, R. (2020, March 25). *Routing to Controller Actions*. Retrieved from Microsoft Docs: <https://docs.microsoft.com/en-us/aspnet/core/mvc/controllers/routing?view=aspnetcore-5.0>
- Smith, S., & Addie, S. (2019, December 5). *Handling requests with controllers in ASP.NET Core MVC*. Retrieved from Microsoft Docs: <https://docs.microsoft.com/en-us/aspnet/core/mvc/controllers/actions>